

Contents

Spark & Scala Practice Lab — 20 Exercises with Step-by-Step Solutions	1
Section A — Scala Essentials (Q1-Q6)	1
Section B — Spark vs Hadoop, RDD Fundamentals (Q7-Q10)	3
Section C — Word Count Labs (Q11-Q12)	4
Section D — Spark Execution Model, Broadcast Variables, Accumulators (Q13-Q17) .	6
Section E — Integration / Synthesis (Q18-Q20)	8

Spark & Scala Practice Lab — 20 Exercises with Step-by-Step Solutions

Covers: Scala essentials (yield, Vector, for-comprehensions, val vs lazy val, traits), Scala vs Python vs Java for Spark, Spark vs Hadoop/MapReduce, RDDs, word count (Scala + PySpark), broadcast variables, accumulators, and the Spark execution model (driver/executors).

Section A — Scala Essentials (Q1-Q6)

Q1. What does yield do in a for-comprehension? Predict the output.

```
val nums = List(1, 2, 3, 4, 5)
val result = for (n <- nums if n % 2 == 0) yield n * 10
```

Solution 1. for (n <- nums if n % 2 == 0) iterates over nums, applying a filter (n % 2 == 0) — this keeps only even elements: 2, 4. 2. yield n * 10 means “don’t just loop — build a new collection.” Each surviving element is transformed by n * 10. 3. Because the source is a List, the result is also a List (the yield preserves the collection type). 4. Final answer: result = List(20, 40). 5. Key rule: for ... yield is an *expression* that produces a value; a plain for (no yield) only produces side effects (like printing) and returns Unit.

Q2. Rewrite this for-comprehension using map/filter directly, and explain the desugaring.

```
val out = for (x <- List(1, 2, 3)) yield x * x
```

Solution 1. The Scala compiler desugars for...yield with a single generator into a .map call. 2. Equivalent code: val out = List(1, 2, 3).map(x => x * x). 3. Both produce List(1, 4, 9). 4. General rule of thumb: - for (x <- xs) yield f(x) → xs.map(f) - for (x <- xs if p(x)) yield f(x) → xs.filter(p).map(f) - for (x <- xs; y <- ys) yield (x, y) → xs.flatMap(x => ys.map(y => (x, y))) 5. Understanding this desugaring matters for Spark, since Spark RDD/Dataset transformations mirror this same map/filter/flatMap chain.

Q3. What is a Vector in Scala, and why might you prefer it over a List?

Solution 1. Vector is an immutable, indexed sequence built as a wide, shallow tree (32-way branching trie). 2. List is a singly linked list: fast prepend/head access (O(1)), but random access by index is O(n). 3. Vector gives effectively constant-time (O(log32 n), practically

~O(1)) random access, updates, and appends/prepends — a good general-purpose default. 4. Example:

```
val v: Vector[Int] = Vector(1, 2, 3, 4)
val updated = v.updated(2, 99) // Vector(1, 2, 99, 4)
println(v(2))                 // fast indexed access
```

5. Rule of thumb: use List when you mostly prepend and traverse sequentially (e.g., recursive algorithms); use Vector when you need frequent random access or a balanced mix of operations.

Q4. Explain the difference between val and lazy val with an example showing evaluation timing.

```
val a = { println("Evaluating a"); 10 }
lazy val b = { println("Evaluating b"); 20 }
println("Before using b")
println(b)
```

Solution 1. A plain val is **strict**: its right-hand side is evaluated immediately, at the point of definition. So "Evaluating a" prints right away when a is declared. 2. A lazy val is **not evaluated until first accessed**. Its initializer is deferred and cached after the first evaluation (evaluated at most once). 3. Trace of execution: - val a = ... → prints Evaluating a immediately. - lazy val b = ... → prints nothing yet. - println("Before using b") → prints Before using b. - println(b) → this first access triggers evaluation → prints Evaluating b, then 20. 4. Full output:

```
Evaluating a
Before using b
Evaluating b
20
```

5. Use lazy val for expensive computations you may never need, or to break initialization-order cycles between fields.

Q5. What is a trait in Scala? Write one and mix it into a class.

Solution 1. A trait is like an interface that can also hold concrete method implementations and fields — Scala's mechanism for multiple inheritance of behavior. 2. Classes/objects "mix in" traits with extends (first one) and with (subsequent ones). 3. Example:

```
trait Greeter {
  def name: String // abstract member
  def greet(): String = s"Hello, $name!" // concrete default method
}

class Person(val name: String) extends Greeter

val p = new Person("Asha")
println(p.greet()) // "Hello, Asha!"
```

4. Traits differ from abstract classes: a class can extend only one class but mix in many traits; traits cannot take constructor parameters (in the same way classes do, prior to

Scala 3 changes) but can define abstract fields.

5. In Spark codebases, traits are commonly used to define reusable contracts (e.g., a Transformer trait) that multiple job classes implement.

Q6. Compare Scala, Python, and Java as languages for writing Spark applications. Give one factual point for each in favor, and one trade-off.

Solution — work through each language: 1. **Scala** - *For*: Spark itself is written in Scala, so Scala Spark code gets the most direct API access and typically the least serialization/interop overhead (no cross-language bridge needed) — often the best raw performance for RDD-heavy or custom logic. - *Trade-off*: Steeper learning curve (functional idioms, implicits, type system) for teams without JVM/functional background. 2. **Python (PySpark)** - *For*: Easiest to learn and most widely used in data science; huge ecosystem (pandas, scikit-learn, matplotlib) for use alongside Spark; DataFrame/SQL API performance is close to Scala's because it runs on the same Catalyst/Tungsten engine. - *Trade-off*: User-defined functions (UDFs) written in plain Python incur serialization overhead crossing the JVM↔Python boundary (via Py4J), which can hurt performance versus native Scala/Java code. 3. **Java** - *For*: Enterprise-friendly, strong typing, mature tooling, integrates naturally into existing Java-based infrastructure. - *Trade-off*: More verbose/boilerplate-heavy than Scala or Python for the same transformations (e.g., lambda syntax exists but the API feels less fluent). 4. **Practical takeaway**: For DataFrame/SQL-based workloads, all three perform similarly since the same engine executes the plan. Scala/Java pull ahead specifically when heavy custom row-by-row logic (UDFs/RDDs) is involved, because Python UDFs cross a process boundary.

Section B — Spark vs Hadoop, RDD Fundamentals (Q7-Q10)

Q7. Is Spark a replacement for Hadoop? Explain step by step.

Solution 1. Break “Hadoop” into its actual components: **HDFS** (storage), **YARN** (resource/cluster management), and **MapReduce** (the original processing engine). 2. Spark is a replacement for the **MapReduce processing engine**, not for Hadoop as a whole ecosystem. 3. Spark commonly *runs on top of* Hadoop infrastructure: it can read/write from HDFS, and it can run as an application on YARN (or on its own standalone cluster manager, or on Kubernetes/Mesos). 4. So the accurate framing: Spark = faster, more general-purpose processing engine that can replace MapReduce, while still cooperating with HDFS/YARN, or running fully independently of Hadoop entirely (e.g., reading from S3, using Kubernetes as the cluster manager). 5. Conclusion: “Spark replaces Hadoop” is a common oversimplification — precisely, **Spark replaces MapReduce**, and can optionally use or bypass the rest of the Hadoop stack.

Q8. Why is Spark generally faster than MapReduce? List the mechanisms.

Solution 1. **In-memory computation**: MapReduce writes intermediate results to disk between every map and reduce stage. Spark keeps intermediate data in memory (RAM) across stages whenever possible, avoiding repeated disk I/O. 2. **DAG execution engine**: Spark builds a Directed Acyclic Graph (DAG) of all the transformations in a job and optimizes the whole pipeline before running it, rather than forcing every job into a rigid two-stage map→reduce pattern. 3. **Lazy evaluation**: Spark doesn't execute transformations until an action is called, which lets it optimize/combine steps (e.g., pipelining a map and filter into a single pass over the data) instead of materializing every intermediate step. 4. **Reduced**

startup/scheduling overhead: Spark reuses JVM executor processes across tasks/stages within an application, whereas classic MapReduce has higher per-task JVM startup overhead. 5. **Caching:** Spark lets you explicitly `persist()/cache()` a dataset in memory for reuse across multiple actions (e.g., iterative machine learning algorithms), something MapReduce has no native equivalent for. 6. Net effect: for iterative or multi-stage workloads, Spark can be an order of magnitude (or more) faster than MapReduce.

Q9. How is data stored/represented in Spark? Explain the concept of partitions and lineage.

Solution 1. Spark's core data abstraction (RDD, and by extension DataFrame/Dataset under the hood) is split into **partitions** — chunks of the dataset distributed across the worker nodes of the cluster. 2. Each partition is processed independently by a task running on an executor, which is what enables parallelism. 3. Data isn't necessarily stored durably by Spark itself — RDDs are usually derived from an external source (HDFS, S3, a database, a local collection) or computed on the fly from transformations. 4. Spark tracks **lineage**: a record of the sequence of transformations used to build an RDD from its original source. This is metadata, not a copy of the data. 5. If a partition is lost (e.g., an executor crashes), Spark uses the lineage graph to **recompute just that partition** from its source/parent RDDs, rather than needing a replicated copy — this is Spark's fault-tolerance mechanism. 6. Optionally, data can be cached in memory (`.cache()/persist()`) across the cluster to avoid recomputation, or written to durable storage (HDFS, S3, Parquet files) as an action.

Q10. What is an RDD? Define it and list its four key properties.

Solution 1. RDD = **Resilient Distributed Dataset** — Spark's fundamental, low-level data abstraction: an immutable, distributed collection of objects that can be processed in parallel. 2. **Resilient** — fault-tolerant via lineage (as in Q9): lost partitions can be recomputed. 3. **Distributed** — data is split into partitions spread across the cluster's worker nodes. 4. **Dataset** — a collection of records/objects (rows, tuples, arbitrary JVM/Python objects). 5. Four defining properties (formally): - A list of **partitions**. - A **compute function** for evaluating each partition. - A list of **dependencies** on other RDDs (the lineage graph). - Optionally, a **partitioner** (for key-value RDDs) and **preferred locations** (for data locality-aware scheduling). 6. RDDs are **immutable**: transformations (`map`, `filter`, `flatMap`, `reduceByKey`, etc.) always produce a *new* RDD rather than mutating the existing one, and are **lazily evaluated** — nothing runs until an action (`collect`, `count`, `saveAsTextFile`, etc.) is called.

Section C — Word Count Labs (Q11-Q12)

Q11. Write a Word Count program in Spark using Scala. Walk through each line.

```
import org.apache.spark.{SparkConf, SparkContext}

object WordCount {
  def main(args: Array[String]): Unit = {
    val conf = new SparkConf().setAppName("WordCount").setMaster("local[*]")
    val sc = new SparkContext(conf)

    val lines = sc.textFile("input.txt")
```

```

val words = lines.flatMap(line => line.split("\\s+"))
val pairs = words.map(word => (word.toLowerCase, 1))
val counts = pairs.reduceByKey(_ + _)

counts.collect().foreach(println)
sc.stop()
}
}

```

Solution — step by step: 1. SparkConf sets the application name and master (local[*] = run locally using all available cores — good for a lab/dev environment). 2. SparkContext (sc) is the entry point/handle to the cluster — every RDD operation goes through it. 3. sc.textFile("input.txt") reads the file and creates an RDD where **each element is one line** of text. 4. flatMap(line => line.split("\\s+")) splits each line into words on whitespace, then **flattens** the resulting arrays of words into a single RDD of individual words (this is why flatMap is used instead of map, which would give an RDD of arrays). 5. map(word => (word.toLowerCase, 1)) transforms each word into a key-value pair (word, 1), lower-cased for case-insensitive counting. 6. reduceByKey(_ + _) groups all pairs by key (the word) and sums the values — this is the “reduce” step, executed in a distributed shuffle across partitions. 7. collect() is the **action** that triggers actual execution of the whole lazy DAG built above, and pulls the final result back to the driver as a local array. 8. foreach(println) prints each (word, count) pair; sc.stop() cleanly shuts down the Spark context.

Q12. Write the equivalent Word Count program in PySpark, and note the key differences from the Scala version.

```

from pyspark import SparkConf, SparkContext

conf = SparkConf().setAppName("WordCount").setMaster("local[*]")
sc = SparkContext(conf=conf)

lines = sc.textFile("input.txt")
words = lines.flatMap(lambda line: line.split())
pairs = words.map(lambda word: (word.lower(), 1))
counts = pairs.reduceByKey(lambda a, b: a + b)

for word, count in counts.collect():
    print(word, count)

sc.stop()

```

Solution — step by step: 1. Structurally identical pipeline to Q11: textFile → flatMap (split lines into words) → map (word → (word, 1)) → reduceByKey (sum counts) → collect (trigger + retrieve results). 2. Key syntactic differences: - Scala uses line.split("\\s+") (regex on whitespace); Python’s line.split() with no args already splits on any whitespace by default. - Scala anonymous functions use =>; Python uses lambda - Scala’s reduceByKey(_ + _) uses placeholder syntax for the combining function; Python needs an explicit lambda a, b: a + b. 3. Under the hood: both versions submit an equivalent logical plan to the same Spark engine. The core RDD transformations (flatMap, map, reduceByKey) are identical in behavior across languages — this is the “same engine, different API surface” idea from Q6. 4. One subtlety: if this were a custom Python **UDF** instead of built-in RDD operations, there would be extra serialization cost crossing the JVM/Python boundary — but plain RDD transformations

like these are implemented efficiently even in PySpark.

Section D — Spark Execution Model, Broadcast Variables, Accumulators (Q13-Q17)

Q13. Walk through a real-time Spark example: counting errors in a stream of log lines by severity level.

```
val logs = sc.textFile("app.log")
val errorLines = logs.filter(line => line.contains("ERROR"))
val severity = errorLines.map(line => (line.split(" ")(2), 1)) // e.g., field 2 = severity code
val counts = severity.reduceByKey(_ + _)
counts.saveAsTextFile("output/error-counts")
```

Solution 1. logs is an RDD of raw log lines, lazily read from app.log. 2. filter keeps only lines containing "ERROR" — a **narrow transformation** (no shuffle needed; each partition is filtered independently). 3. map extracts a severity/category token from each line and pairs it with 1, preparing for aggregation. 4. reduceByKey is a **wide transformation** — it requires a **shuffle**, since values for the same key may live on different partitions/nodes and must be brought together. 5. saveAsTextFile is the **action** — until this call, nothing above has actually executed; Spark has just been building a DAG of the plan. 6. This mirrors realistic production use: filter → extract → aggregate → persist results, typically run repeatedly (e.g., hourly) over rotating log files, or continuously via Spark Structured Streaming for true real-time processing.

Q14. What is a broadcast variable? Why and when should you use one? Provide code.

```
val severityMap = Map("E1" -> "Critical", "E2" -> "Warning", "E3" -> "Info")
val broadcastMap = sc.broadcast(severityMap)

val labeled = severity.map { case (code, cnt) =>
  (broadcastMap.value.getOrElse(code, "Unknown"), cnt)
}
```

Solution 1. Problem it solves: if severityMap were referenced directly inside a closure passed to map, Spark would serialize and ship a **copy of that map to every task** on every executor — wasteful for a large read-only lookup table shared across many tasks. 2. sc.broadcast(severityMap) sends the value to each **executor node once** (not once per task) and caches it there; tasks on that executor then read it via .value from local memory. 3. Use cases: shipping a lookup table, a small reference dataset, or a machine learning model's parameters to all workers efficiently. 4. Key properties: broadcast variables are **read-only** once created (you can't update them and have that change propagate) and are most beneficial when the data is reused across **many tasks**, and large enough that per-task shipping would be wasteful. 5. In the example, instead of every task carrying its own copy of severityMap, all tasks on a given executor share the single broadcast copy, reducing network and memory overhead.

Q15. What is an accumulator? Demonstrate with a malformed-record counter.

```
val malformedCount = sc.longAccumulator("MalformedRecords")

val parsed = logs.map { line =>
  val parts = line.split(",")
  if (parts.length != 5) {
    malformedCount.add(1)
    None
  } else Some(parts)
}.flatMap(x => x)

parsed.count() // action triggers evaluation
println(s"Malformed records: ${malformedCount.value}")
```

Solution 1. An **accumulator** is a variable that worker tasks can only **add to** (write-only from the executors' perspective); only the **driver** can read its final value via `.value`. 2. It's designed for safe, distributed counters/sums — e.g., counting errors, summing a metric — across many parallel tasks without needing a shuffle/reduce operation. 3. In the example, `malformedCount.add(1)` is called inside the map closure every time a malformed line is found; each task accumulates its own partial count locally. 4. Spark automatically and safely merges all tasks' partial accumulator values back to the driver when the stage completes. 5. Critical caveat: `parsed.count()` (an **action**) is required to actually trigger execution — accumulator updates inside transformations only happen when the lazy DAG is executed. Also, if a stage is **recomputed** (e.g., due to a retry or lineage-based recovery), accumulator updates inside transformations can be double-counted — accumulators are guaranteed accurate only for updates inside **actions**, so use them cautiously for monitoring/debugging rather than business-critical counts.

Q16. Explain, step by step, how Spark executes a program on a cluster once an action is called.

Solution 1. **Build the DAG:** As transformations (`map`, `filter`, `reduceByKey`, etc.) are called, Spark just records them lazily as a logical DAG of RDD dependencies — nothing runs yet. 2. **Action triggers execution:** When an action (`collect`, `count`, `saveAsTextFile`, etc.) is called, the **driver** program's DAG scheduler kicks in. 3. **Divide into stages:** The DAG scheduler splits the DAG into **stages**, breaking at each point where a **shuffle** (wide dependency) is required, e.g., before/after a `reduceByKey`. 4. **Divide stages into tasks:** Each stage is broken into **tasks** — one task per partition. Tasks are the actual units of work sent to executors. 5. **Task scheduling:** The **Task Scheduler** sends tasks to available **executors** on worker nodes, ideally choosing executors that already hold the relevant data partition in memory/local disk (data locality). 6. **Execution on executors:** Each executor runs its assigned tasks in threads, applying the compute chain for that partition, and either produces a shuffle output for the next stage, or (for the final stage) returns results to the driver. 7. **Result assembly:** The driver collects/aggregates task results (e.g., for `collect()`, it gathers all partitions; for `count()`, it sums per-partition counts) and returns the final value to user code.

Q17. What are the roles of the driver and the executors in a Spark application? How do they communicate?

Solution 1. **Driver:** - Runs the `main()` method of the Spark application and creates the Spark-Context/SparkSession. - Converts user code into a logical DAG, splits it into stages/tasks (via the DAG Scheduler and Task Scheduler), and schedules those tasks onto executors. - Collects results from executors for actions like `collect()`, and maintains the overall application state (e.g., accumulator values). 2. **Executors:** - JVM (or Python worker, for PySpark) processes launched on worker nodes at the start of the application. - Actually execute the tasks assigned to them: run the transformation logic against their assigned partitions. - Store data for caching/persistence (`.cache()`) in their own memory/disk, and report status/results back to the driver. 3. **Communication:** The driver and executors communicate over the network — the driver sends serialized task code + broadcast variables to executors, and executors send back task status, metrics, and results (or shuffle data to each other directly during wide-dependency stages). 4. **Cluster manager's role (context):** A cluster manager (YARN, Kubernetes, Mesos, or Spark's own standalone manager) is responsible for actually allocating the worker node resources on which the driver requests executors to be launched — it sits “underneath” the driver/executor relationship. 5. Practical implication: if an executor dies, the driver detects the failure and **reschedules its tasks** on another executor, recomputing lost partitions from lineage (tying back to Q9/Q10's fault-tolerance model).

Section E — Integration / Synthesis (Q18-Q20)

Q18. Explain the difference between RDD transformations and actions, and classify these five operations: `map`, `filter`, `reduceByKey`, `collect`, `count`.

Solution 1. **Transformations** are lazy — they define a new RDD from an existing one but don't execute anything immediately; they just extend the DAG. 2. **Actions** trigger actual computation across the cluster and either return a value to the driver or write output to storage. 3. Classification: - `map` → transformation (narrow — no shuffle). - `filter` → transformation (narrow — no shuffle). - `reduceByKey` → transformation (wide — requires a shuffle). - `collect` → action (returns all data to the driver as a local collection). - `count` → action (returns the number of elements to the driver). 4. Why the distinction matters: chaining many transformations costs nothing until an action runs, which is why Spark can optimize the whole pipeline (e.g., combine filters and maps into a single pass) before it ever touches data.

Q19. Why does lazy evaluation matter for performance in Spark? Illustrate with a scenario where an unnecessary computation is avoided.

```
val raw = sc.textFile("bigfile.txt")
val filtered = raw.filter(_.contains("ERROR"))
val firstFive = filtered.take(5)
```

Solution 1. Because `filter` is lazy, Spark does **not** scan and filter the entire (potentially huge) file up front. 2. `take(5)` is an action, but it's a special one: Spark will process partitions **incrementally**, stopping as soon as it has collected 5 matching results — it does not necessarily need to touch every partition of `bigfile.txt`. 3. Contrast with an eager model: if `filter` executed immediately, Spark would be forced to scan and filter the *entire* file before `take(5)` could even be called, wasting effort on data whose results are never used. 4. This laziness-plus-optimization combination (deferring work, then planning around only what's actually needed for the requested action) is a core reason Spark can be efficient even on very

large datasets — it ties directly back to the “why Spark is faster” reasoning in Q8.

Q20. Design (in pseudocode/Scala) a small Spark job that uses both a broadcast variable and an accumulator together, and explain the role of each.

```
// Reference set of "VIP" customer IDs – small, reused across all tasks
val vipIds = Set("C001", "C045", "C102")
val vipBroadcast = sc.broadcast(vipIds)

// Counter for how many VIP transactions we find
val vipTransactionCount = sc.longAccumulator("VipTransactions")

val transactions = sc.textFile("transactions.csv")

val flagged = transactions.map { line =>
  val cols = line.split(",")
  val customerId = cols(0)
  val isVip = vipBroadcast.value.contains(customerId)
  if (isVip) vipTransactionCount.add(1)
  (customerId, isVip)
}

flagged.count() // action: triggers execution
println(s"VIP transactions found: ${vipTransactionCount.value}")
```

Solution — step by step:

- 1. Broadcast variable (vipBroadcast):** The set of VIP customer IDs is small but needs to be checked against **every single transaction row**, across potentially many tasks/executors. Broadcasting it once per executor avoids re-shipping the set with every task’s closure — exactly the efficiency case from Q14.
- 2. Accumulator (vipTransactionCount):** We want a simple running total of “how many transactions were flagged as VIP,” aggregated across all distributed tasks, without needing a full reduce-ByKey/shuffle just to get one number — this is the classic use case from Q15.
- 3. Execution flow:** Because everything above is a lazy transformation until `flagged.count()` (an action) is called, nothing runs until that line — matching the lazy-DAG behavior from Q16/Q19.
- 4. Reading the result:** After the action completes, `vipTransactionCount.value` is safely readable **on the driver only**, giving the final merged count from all executors.
- 5. Why this combination is realistic:** This pattern — a broadcasted reference/lookup dataset plus an accumulator for monitoring/metrics — is extremely common in real Spark pipelines (e.g., fraud flagging, log classification, data-quality checks), tying together nearly every concept covered in this lab: RDDs, transformations vs. actions, the driver/executor model, and Spark’s two special-purpose distributed variables.

End of practice set — 20 questions covering Scala fundamentals and the full Spark execution/optimization model.